COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/11/2026	**DATE SUBMITTED:**	09/11/2026
NAME:	Milan Kherwin Emmanuel, Moradillo Eunice, Nieles Cenndy, Realubit Seth Andrew	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 Using Sorting Algorithms

1. PROCEDURE OUTPUT

1. Explain the quick sort, shell sort, and merge sort types of sorting algorithms.

- **Quick Sort:** follows the algorithm of divide and conquer using a pivot, i, and j along with its 3 conditions. These 3 conditions swap the i and j variables depending whether the j > pivot, j < pivot, and j == pivot. Additionally, it has a time complexity of O(n log n) as the best case and $O(n^2)$ as the worst case.

- **Shell Sort:** is an unstable version of insertion sort due to the gap which can prove to be unpredictable. It works by the gap being divided in halves and compares the elements that are separated by this gap, sorts these elements and rinse and repeat. Additionally, the time complexity is O(n log n) as the best case and O(N^2) worst case.

- **Merge Sort:** also follows a divide-and-conquer kind of algorithm but instead of depending on the pivot, it cuts the data into the smallest pieces before getting arranged. It follows the procedure of cutting down the unsorted array in half until individual elements merge back into a sorted order and repeat the process until it becomes a sorted list. Additionally, it follows the time complexities of O(n log n) as its worst and best case.

2. Give simple sample programs in C++ that use the above sorting algorithms. Use a user input of 10 integer values in your example elements in an array to be sorted. Explain how the programs work.

Shell Sort - Embedded System and Microcontrollers

In microcontrollers like an Arduino, STM32, or PIC microchip, memory architecture is tightly constrained with RAM often measured in bytes rather than gigabytes. Advanced sorting algorithms like Quick Sort rely heavily on recursion, which constantly pushes data onto the stack memory and risks a devastating stack overflow crash. **Shell sort** bypasses this entirely by using simple while and for loops instead of recursion, making its memory footprint 100% predictable. In automotive sensors or industrial IoT monitors, data is collected in small batches directly inside hardware registers, allowing Shell sort to cleanly organize these buffers without requesting a single byte of temporary system RAM.

Explanation:

The single biggest constraint on a microcontroller isn't speed, it's memory. These devices often have only a few kilobytes of RAM total, so any algorithm that eats into that budget unpredictably is dangerous. Shell sort avoids this problem entirely because it's built from simple loops, not recursion, so it never adds hidden memory overhead. It uses a small, fixed amount of extra memory (O(1)) no matter how much data it's sorting. So the priority in embedded systems is reliability over raw speed, a slightly slower sort that never crashes beats a faster one that might. That's why shell sort is the better fit, it trades some sorting speed (its time complexity isn't as strong as quick sort's) for guaranteed memory safety, which matters far more when you're running on a chip with just a few kilobytes of RAM.

Quick sort - Virtual Memory and Desktop Operating Systems

Desktop and mobile operating systems utilize virtual memory management, which handles data in chunks called memory pages. **Quick Sort** interacts incredibly well with this architecture because it exhibits excellent cache locality, meaning it tends to read and write memory addresses that are physically close to each other as it partitions data around a pivot point. This localized behavior prevents the computer's CPU from constantly fetching data from the slower main RAM, keeping the sorting operations entirely within the ultra-fast CPU cache and drastically reducing hardware wait times.

Explanation :

The main priority for a desktop or mobile operating system is speed, since it's constantly sorting large amounts of data (files, processes, search results) while users expect near instant responses. Quick sort fits this because of how it partitions data because it works on nearby sections of memory at a time, which lines up naturally with how CPU caches are designed to grab chunks of nearby data at once. This means the CPU can keep reusing data already sitting in its fast cache instead of repeatedly fetching from much slower RAM, cutting down wait time significantly. Its average-case time complexity of O(n log n) makes it one of the fastest general sorting methods for large datasets. So while quick sort doesn't have the strict memory safety that shell sort offers, that trade off makes sense here, since desktop systems have plenty of RAM to spare and speed matters far more than staying within a tiny memory budget.

Merge Sort - Big Data and External Sorting Systems

In big data processing and database systems, datasets often grow far larger than what fits into RAM at once (e.g., think sorting a multi-gigabyte log file or a database table with millions of records). Merge sort handles this naturally because it breaks the data into smaller chunks, sorts each chunk independently, and then merges the sorted pieces back together. This "divide, sort, merge" pattern doesn't require random access to the whole dataset at once, so chunks can be read from disk, sorted, written back out, and merged sequentially or a technique called external sorting.

Explanation:

The main priority of big data and distributed systems is not raw memory or the CPU cache, it's handling data that's too large to fit in one location and combining the results from multiple sources. Merge sort fits this because it processes the data independently, in continuous chunks rather than needing random access to the entire data set like quick sort does. This makes it the best option out of the three because of its external sorting, where data is split into individual pieces before getting merged. It is also divisible due to its natural process of sorting in a different place before getting put back together. It's consistent time complexity of O(n log n), unlike quick sort which can decay on unlucky pivots, so merge sort is the dependable choice when combining sorted outputs from different sources rather than sorting everything in one place at once.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.